

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5112

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

MARKS: 25

Annexure A

TOTAL

CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

*I _E .Lakshmi Narayanan(192524249)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E .Lakshmi Narayanan

Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

Problem Understanding

The financial organization needs an IDS that can process **150,000 packets/second** while satisfying:

- Latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- High traffic volume
- Limited CPU resources

A purely anomaly-based IDS may provide better detection of unknown attacks but generally requires more CPU. A purely signature-based IDS is faster but may miss zero-day attacks.

Proposed Solution: Hybrid IDS

A **hybrid IDS** combining signature-based and lightweight anomaly detection is the best solution.

Traffic $\rightarrow$ Signature Filter $\rightarrow$ Lightweight Anomaly Analysis $\rightarrow$ Alert/Allow

Configuration

Component	Configuration
Signature detection	Known attack signatures
Anomaly detection	Lightweight statistical behavior analysis
Traffic rate	150,000 packets/sec

Component	Configuration
Target latency	$\leq 5 \text{ ms}$
Target accuracy	$\geq 96\%$
False positives	$\leq 1.5\%$
CPU optimization	Parallel processing + rule prioritization

Constraint Analysis

At 150,000 packets/sec, the average processing time available per packet is:

$$1500001 = 6.67 \mu\text{s}$$

Therefore, the IDS must use highly optimized packet processing rather than performing expensive analysis on every packet.

The **signature engine handles most traffic first**. Only suspicious packets are forwarded to the anomaly engine. This reduces CPU consumption.

Trade-off

- Signature-based detection → **fast and low CPU**, but weaker against unknown attacks.
- Anomaly-based detection → **better zero-day detection**, but higher CPU usage and false positives.
- Hybrid detection → balances both.

Expected Performance

With optimized parallel processing:

- Latency: $\approx 3\text{--}4 \text{ ms/packet}$
- Accuracy: $\geq 96\%$
- False positive rate: $\leq 1.5\%$
- Traffic handling: **150,000 packets/sec**

Justification

The hybrid approach is optimal because signature matching provides fast detection for known attacks, while lightweight anomaly analysis improves detection of previously unseen behavior. Rule prioritization, packet filtering and selective anomaly analysis reduce CPU usage.

2. Worm Containment Strategy under Network Availability Constraints

Problem Understanding

The enterprise is experiencing rapid worm propagation. The solution must:

- Keep downtime $\leq 1\%$
- Detect and contain worms within **2 minutes**
- Protect multiple locations
- Work with limited centralized control
- Maintain business continuity

Proposed Solution

Use **distributed IDS/IPS + firewalls + network segmentation**.

Architecture

Internet → Firewall → IDS/IPS → Network Segments

Each branch/location has its own:

- Firewall
- IDS/IPS
- VLAN/network segmentation
- Local containment rules

Containment Process

Step 1: Detection

IDS detects abnormal scanning, repeated connection attempts and unusual traffic.

Step 2: Verification

IPS checks the traffic against worm signatures and behavioral rules.

Step 3: Isolation

Only the infected VLAN, host or subnet is isolated instead of shutting down the entire network.

Step 4: Firewall Blocking

Firewall blocks the worm's source IP, destination ports or malicious traffic pattern.

Step 5: Recovery

After cleaning and verification, the isolated system is reconnected.

Time Calculation

Required containment time:

$T_{\text{detection}} + T_{\text{verification}} + T_{\text{isolation}} \leq 2 \text{ minutes}$

Example:

- Detection = 30 seconds
- Verification = 20 seconds
- Isolation = 30 seconds

$30 + 20 + 30 = 80 \text{ seconds}$

Therefore:

$80 < 120$ seconds

The 2-minute constraint is satisfied.

Availability

Instead of shutting down the complete enterprise network, only affected segments are isolated.

For example, if there are 20 network segments and only 1 is infected:

$20 \times 100 = 5\%$

Only the affected segment is temporarily unavailable rather than the entire organization.

Trade-off

Complete network isolation

- Better containment
- Very high downtime
- Poor business continuity

Selective segmentation

- Slightly greater risk of spread
- Much lower downtime
- Better business continuity

Justification

A distributed containment strategy is suitable because centralized control is limited. Local IDS/IPS and firewalls can independently detect and isolate infected systems. Network segmentation prevents the worm from moving freely between departments and locations. Selective isolation minimizes downtime while achieving containment within 2 minutes.

3. Firewall Rule Optimization under Performance Constraints

Problem Understanding

The firewall has **10,000+ rules**, causing performance degradation. The requirements are:

- Throughput $\geq$ **800 Mbps**
- Minimize rule-processing time
- Maintain strict access control
- No additional hardware

The main problem is inefficient rule evaluation.

Sources of Inefficiency

1. Duplicate rules
2. Overlapping rules
3. Shadowed rules
4. Unnecessary rules
5. Poor rule ordering
6. Repeated IP addresses and ports
7. Broad rules placed before specific rules

Proposed Optimization

1. Remove Redundant Rules

If two rules perform the same function, keep only one.

Example:

Rule 100: Allow 192.168.1.10 $\rightarrow$ Server A

Rule 500: Allow 192.168.1.10 $\rightarrow$ Server A

Rule 500 is redundant.

2. Rule Aggregation

Instead of:

Allow 192.168.1.1

Allow 192.168.1.2

Allow 192.168.1.3

Allow 192.168.1.4

use an appropriate subnet/network rule where policy permits.

3. Prioritize Frequent Rules

Frequently used rules should be placed near the top.

Example:

High-frequency web traffic

↓

Common business applications

↓

Less frequent traffic

↓

Rare traffic

↓

Default DENY

4. Remove Shadowed Rules

Example:

Rule 1: DENY all traffic

Rule 2: ALLOW HTTP traffic

Rule 2 will never be reached. Therefore, the rule ordering must be corrected.

Example Calculation

Suppose optimization reduces rules from:

10,000→4,000

Reduction:

$10,000 - 4,000 = 6,000$

Percentage reduction:

$\frac{6,000}{10,000} \times 100 = 60\%$

Thus, the firewall performs substantially fewer rule comparisons.

Expected Result

Parameter	Before	After
Rules	10,000+	~4,000
Redundancy	High	Low
Rule searches	High	Reduced
Processing time	High	Reduced
Throughput	<800 Mbps	≥800 Mbps target
Security	Maintained	Maintained

Justification

Rule aggregation, redundancy elimination and intelligent prioritization reduce the number of comparisons required for each packet. The final rule set still follows strict access-control policies and retains a default-deny rule. Therefore, performance improves without requiring additional hardware or compromising security.

4. IPS Deployment under Resource and Accuracy Constraints

Problem Understanding

The organization requires an IPS that:

- Blocks $\geq 90\%$ of known attacks
- False positives $\leq 3\%$
- Signature memory $\leq 1\text{ GB}$
- Network speed = 500 Mbps

The major challenge is selecting important signatures while staying within memory limits.

Proposed IPS Configuration

Use a **risk-based signature selection strategy**.

Instead of loading every available signature, select:

1. Critical vulnerability signatures
2. Frequently exploited signatures
3. High-severity malware signatures
4. Signatures relevant to the organization's applications
5. Recent attack signatures

Signature Optimization

Suppose the original signature database requires:

1.5 GB

but the available memory is:

1 GB

Remove low-risk and irrelevant signatures and compress/optimize the signature database to:

850 MB

Therefore:

850MB<1GB

The memory constraint is satisfied.

Traffic Filtering

Traffic should first be filtered based on:

- Protocol
- Port
- Source/destination
- Known trusted traffic
- Suspicious patterns

This prevents unnecessary signature processing.

Adaptive Updates

The IPS should regularly update signatures based on:

- Newly discovered attacks
- Threat intelligence
- Frequently observed attacks
- Organization-specific vulnerabilities

Old, unused signatures can be disabled.

Expected Performance

Requirement	Target
Attack blocking	$\geq 90\%$
False positive rate	$\leq 3\%$
Signature memory	≤ 1 GB
Network speed	500 Mbps

Example expected result:

- Known attacks blocked = **93%**
- False positives = **2.5%**
- Signature memory = **850 MB**
- Network speed supported = **500 Mbps**

All constraints are satisfied.

Trade-off

Loading every signature increases detection capability but consumes memory and processing power.

Selecting only high-risk signatures reduces resource consumption but may miss some low-frequency attacks.

The risk-based approach provides the best balance.

Justification

The proposed IPS uses prioritized signatures, traffic filtering and adaptive updates. This keeps signature memory below 1 GB while focusing resources on high-risk attacks. A target blocking rate of 93% and false positive rate of 2.5% satisfies the required constraints.

5. Malware Detection using Behavioral Analysis under System Constraints

Problem Understanding

The company needs behavioral malware detection that:

- Uses $\leq 15\%$ **endpoint CPU**
- Detects malware within **10 seconds**
- Detects unknown/zero-day threats
- Uses limited log storage

Traditional signature-based detection cannot reliably detect completely new malware. Therefore, behavioral analysis is required.

Proposed Lightweight Architecture

Program Execution → Behavior Monitoring → Feature Extraction → Risk Scoring → Alert/Block

Monitored Behaviors

Instead of monitoring everything, monitor high-risk activities such as:

- Unusual file modifications
- Registry changes
- Suspicious process creation
- Privilege escalation
- Network connections
- Mass file encryption
- Abnormal process injection

Lightweight Feature Extraction

For every process, calculate a risk score.

Example:

Behavior	Score
Suspicious process creation	+20
Registry modification	+15
Unusual network connection	+20
Mass file modification	+30
Privilege escalation	+25

Suppose a process produces:

$$20+15+20+30=85$$

If the malware threshold is:

$$\text{Risk Score} \geq 70$$

then:

$$85 > 70$$

Therefore, the process is classified as suspicious.

CPU Optimization

To maintain CPU usage $\leq 15\%$:

- Monitor only high-risk events
- Avoid continuous full-system scanning
- Use event-driven monitoring
- Analyze suspicious processes instead of every process
- Use lightweight statistical models

Example:

CPU Usage=12%

Therefore:

$12\% < 15\%$

Constraint satisfied.

Detection Time

Example:

- Behavior collection = 4 seconds
- Feature extraction = 2 seconds
- Risk analysis = 1 second
- Alert/action = 1 second

$4+2+1+1=8$ seconds

Therefore:

$8 < 10$ seconds

The detection-time constraint is satisfied.

Limited Log Storage

Instead of storing every event, store only:

- High-risk events
- Detection results
- Process ID
- Timestamp
- Risk score
- Important behavioral indicators

This significantly reduces storage requirements.

Trade-off

Approach	Advantage	Disadvantage
Deep monitoring	High accuracy	High CPU
Minimal monitoring	Low CPU	May miss attacks
Selective behavioral monitoring	Good balance	Requires careful feature selection

Justification

The proposed lightweight behavioral detection system focuses only on security-critical activities. Event-driven monitoring reduces CPU consumption, while risk scoring allows unknown malware to be identified based on behavior rather than known signatures. With an example CPU usage of 12% and detection time of 8 seconds, the system satisfies the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided